High Accuracy Sign Attention Network for Traffic Sign Perception
Technical Report

Abstract

Traffic sign recognition supports driver assistance and autonomous driving by helping vehicles interpret road rules. Small traffic signs remain difficult to recognize when visual detail is limited or image conditions change. Existing methods can suffer from low accuracy, low efficiency, and low generalizability under these conditions. This paper proposes the High Accuracy Sign Attention Network (HASAN), a detector design that combines selective convolution, channel gating, spatial pooling, and spatial attention. We examine training parameters and recognition under changed image conditions. Our results show that HASAN achieves 97.997% mAP@0.5 on GTSRB, exceeding the reported 96.800% of the strongest state-of-the-art comparison method by 1.197 percentage points, a relative improvement of 1.237%.
Keywords: Traffic sign recognition; Small object detection; Attention mechanism; Convolutional neural network; Driver assistance systems;

1 Introduction

1.1 Background

Traffic signs communicate rules and warnings that driver assistance systems must recognize reliably. Existing approaches fall into four families: handcrafted feature methods, region proposal methods, direct prediction methods, and transformer prediction methods. Handcrafted methods use color, shape, and manually designed descriptors [1]. Region proposal methods generate candidate regions and combine features at multiple resolutions [2,3]. Direct prediction methods, including You Only Look Once (YOLO), estimate objects in one processing path [4,5]. Transformer prediction methods use attention and set prediction [6]. Traffic sign specific studies address difficult weather and small sign recognition [7,8]. Reliable recognition must preserve the visual details that distinguish similar signs while handling changes in illumination, viewing angle, and regional design. Cropped sign benchmarks provide a controlled measure of class recognition [9], while globally collected images expose wider appearance variation [10].

1.2 Limitations

Existing traffic sign methods face three primary challenges: low accuracy, low efficiency, and low generalizability under difficult operating conditions.

First, existing methods often suffer from low accuracy on small or degraded signs. Handcrafted features lose useful detail under changing illumination, while region proposal methods depend on the quality of candidate regions and extracted features [1,2,3]. These weaknesses motivate more selective feature processing.

Second, existing detectors face efficiency limitations when computational resources are limited. Convolutional and transformer methods must balance recognition quality against model complexity and prediction cost [4,5,6]. Traffic sign refinements also need to recover small object detail without excessive processing [7,8].

Third, existing methods often struggle to generalize to unfamiliar images. Changes in sign design, scene conditions, and dataset composition can reduce recognition accuracy [9,10]. This motivates evaluation with changed images as well as the original benchmark.

1.3 Contributions

This paper makes three main contributions.

First, we propose the High Accuracy Sign Attention Network (HASAN), a detector design that combines selective convolution with channel gating and spatial attention. Partial channel processing extracts local features, gating controls their contribution, and attention refines the pooled spatial context before prediction.

Second, we varied parameters in terms of five factors: kernel configuration, learning rate, learning rate scheduler, optimizer, and batch size. Each comparison changes one factor while holding the remaining settings fixed, allowing its effect on mAP, recall, and inference time to be examined separately.

Third, we varied the evaluation dataset. We evaluate HASAN on Chinese Traffic Signs (CTS), the German Traffic Sign Recognition Benchmark (GTSRB), and the Tsinghua-Tencent 100K (TT100K) dataset. This comparison examines detection performance across different traffic sign datasets.

Together, the study defines the HASAN feature processing design, identifies parameter settings achieving 97.997% mAP@0.5 on GTSRB compared with 96.800% for the strongest comparison method, and examines performance across CTS, GTSRB, and TT100K. The GTSRB mAP improvement is 1.197 percentage points, or 1.237% relative.

2 Related Work

We review four families of traffic sign methods and discuss how their limitations in accuracy, efficiency, and generalizability motivate this study.

2.1 Handcrafted Feature Methods

Handcrafted methods identify signs from color, shape, and manually designed descriptors, then classify them with conventional classifiers. Recent reviews describe these pipelines as an important starting point for traffic sign recognition [1].

Limitation: These methods have low accuracy when lighting or image quality changes. Fixed descriptors can confuse visually similar signs and do not adapt easily to regional appearance changes [1].

2.2 Region Proposal Methods

Region proposal methods first locate candidate objects, then refine their classes and boxes. Faster Region Based Convolutional Neural Networks generate learned proposals [2], while feature pyramid networks combine representations at several resolutions to support objects of different sizes [3].

Limitation: These methods suffer from low efficiency when proposal generation adds computation. Candidate generation and refinement add processing, while small signs can still be lost when their features are weak [2,3].

2.3 Direct Prediction Methods

Direct prediction methods estimate classes and locations in one processing path. YOLOv7 improves trainable detection components, while YOLOv10 reduces redundant computation and postprocessing [4,5]. Traffic sign adaptations refine feature extraction for weather variation and small targets [7,8]. Improved YOLOv7 strengthens small-sign features through revised spatial pyramid pooling and attention-guided upsampling, while normalized Wasserstein distance reduces localization sensitivity for small objects [11]. CPB-YOLOv8 combines a convolution–transformer feature extractor with a bidirectional feature pyramid, an additional high-resolution detection layer, and an adjusted box regression loss to improve multiscale detection [12]. Tversky-YOLO introduces asymmetric similarity into backbone feature gating and prototype-based classification to distinguish visually similar traffic signs [13].

Limitation: These methods still face accuracy and efficiency limitations in difficult conditions. The Improved YOLOv7 study identifies recognition speed as an area needing further improvement [11]. In CPB-YOLOv8, the high-resolution detection layer improves small-target detection but increases computational load [12]. Tversky-YOLO remains less reliable for extremely small or heavily occluded signs [13]. These findings motivate selective feature processing that preserves small-sign detail while controlling computation.

2.4 Transformer Prediction Methods

The Real Time Detection Transformer (RTDT) combines convolutional features with attention and set prediction [6]. Its hybrid encoder is designed to improve the balance between detection accuracy and computation.

Limitation: Transformer methods face efficiency limitations when attention processing is costly. Efficient encoder design and query selection are needed to keep prediction practical, and benchmark detection accuracy alone does not establish generalizability to new traffic sign domains [6].

3 Methodology

3.1 Overview

Figure 1 shows the overview of HASAN. First, image preparation resizes and normalizes the input. Second, feature refinement applies selective convolution, channel gating, spatial pooling, and spatial attention. Third, sign prediction converts the refined features into classes and boxes and selects the final detections. Selective processing preserves local detail, gating controls the feature contribution, and attention emphasizes useful spatial context.

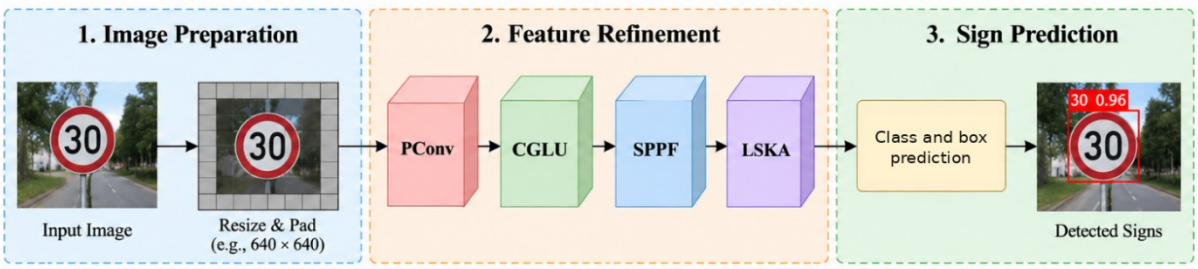


Figure 1. HASAN overview. Resize & Pad denotes resizing followed by padding. The example input dimensions are measured in pixels. The output label shows the sign category and an illustrative confidence score.

3.2 Image Preparation

The input is a red, green, and blue (RGB) image. It is resized to the model input size while preserving its aspect ratio, then padded to a square. Pixel values are divided by 255 to place them between zero and one. During training, the same resizing and padding are applied to the annotated boxes. The scale and padding offsets are stored so predicted boxes can be mapped back to the original image. We denote the prepared image by X_0. The detector backbone maps it to the feature tensor X_1 entering the refinement sequence.

3.3 Feature Refinement

Feature refinement follows four operations in order: partial convolution, channel gating, spatial pooling, and spatial attention. Figures 2 to 5 show the corresponding module connections. The tensors X_1, X_2, X_3, X_4, and X_5 identify successive representations, so each module receives the output of the previous module. The descriptions follow the same sequence as the diagrams. Subscripts p and r denote processed and remaining channels. Function arguments specify the tensor being transformed. In the illustrations, H and W denote feature map height and width, and C denotes the number of channels. Conv denotes convolution, and k × k specifies its kernel size. Concat joins feature maps along the channel dimension. A feature map is a tensor of learned responses. Arrows show the direction of data flow, while the stacked sheets represent feature channels schematically.

3.3.1 Partial Convolution

Partial Convolution (PConv) filters a subset of feature channels and passes the remaining channels through unchanged [14]. Figure 2 shows the two paths. We split X_1 along its channel dimension into X_{1p} and X_{1r}, apply a k × k convolution to X_{1p}, and concatenate it with the unmodified X_{1r}. Equation (1) produces X_2 while preserving the spatial dimensions. Concat denotes channel concatenation.

$$X_2 = Concat\left(Conv\left(X_{1p}\right),\ X_{1r}\right) \quad (1)$$

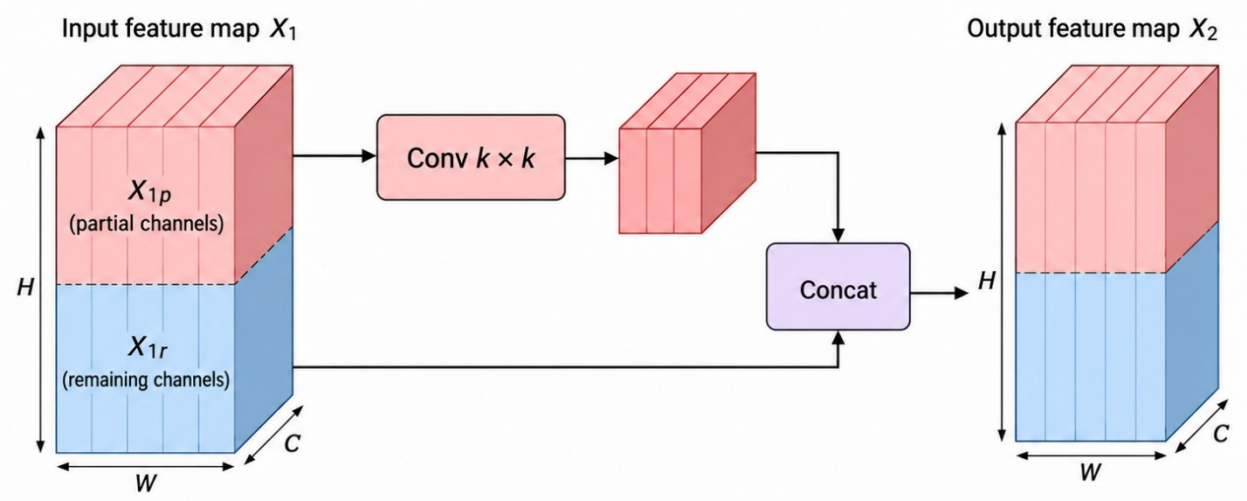


Figure 2. Partial convolution.

3.3.2 Convolutional Gated Linear Unit

Convolutional Gated Linear Unit (CGLU) processing controls which feature responses contribute to the output [15]. As shown in Figure 3, two learned projections form parallel branches from the incoming features. One branch applies depthwise convolution (DWConv) followed by a Gaussian Error Linear Unit (GELU) activation to extract local patterns. The other branch provides a gate that modulates these patterns through elementwise multiplication. An output projection mixes the gated features, and a residual connection adds the original input to produce the refined output. DWConv filters each channel separately. Projection denotes a learned transformation of the channel features. The circled multiplication symbol denotes elementwise gating, and the circled plus symbol denotes elementwise residual addition.

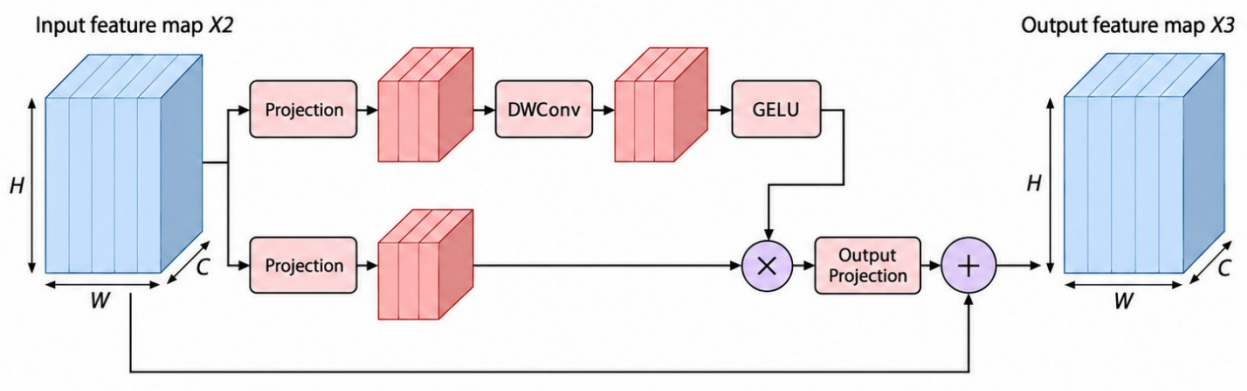


Figure 3. Convolutional gated linear unit.

3.3.3 Spatial Pyramid Pooling Fast

Spatial Pyramid Pooling Fast (SPPF) aggregates context at different spatial scales. As shown in Figure 4, a 1 × 1 convolution first projects the incoming features. Three successive 5 × 5 max pooling operations then expand the region represented by each feature location. Each pooling operation uses a stride of one and padding to preserve the spatial dimensions. The projected input and the outputs of all three pooling operations are concatenated along the channel dimension. A final 1 × 1 convolution combines them into the output features. In Figure 4, MaxPool denotes max pooling, which retains the largest response within each pooling window. The label 4C denotes the combined width of four concatenated tensors, each with C channels at that point in the diagram. A 1 × 1 convolution mixes channel information at each spatial location.

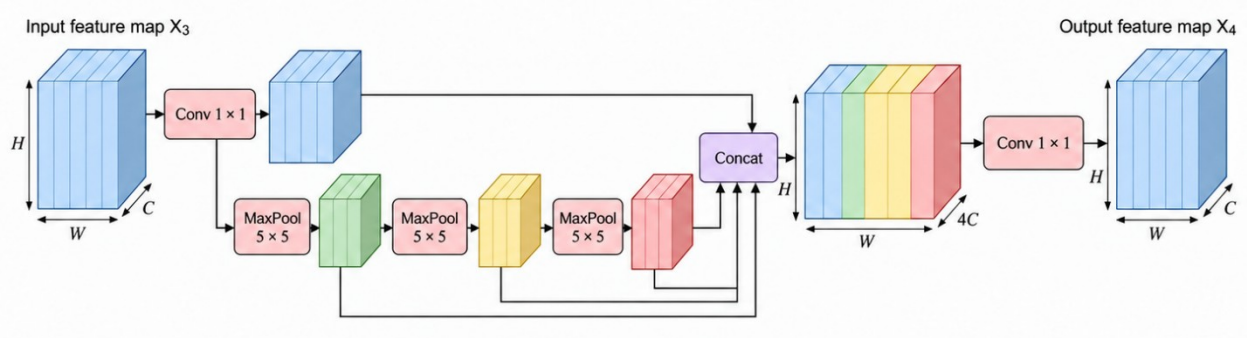


Figure 4. Sequential spatial pyramid pooling.

3.3.4 Large Separable Kernel Attention

Large Separable Kernel Attention (LSKA) uses spatial context to weight the pooled features [16]. As shown in Figure 5, horizontal and vertical depthwise filters first capture spatial relationships along each direction. Dilated horizontal and vertical filters then extend the region covered by the operation. A 1 × 1 convolution combines the filtered responses to form an attention map. Elementwise multiplication of this map with the incoming features emphasizes relevant spatial responses. In Figure 5, the filter suffixes h and v denote horizontal and vertical orientations, and d denotes dilation, which spaces filter samples farther apart. A denotes the attention map, and the circled multiplication symbol applies its weights elementwise to the incoming features.

The factorized filters capture broad spatial context while preserving the feature grid. The resulting features are passed to the sign prediction stage.

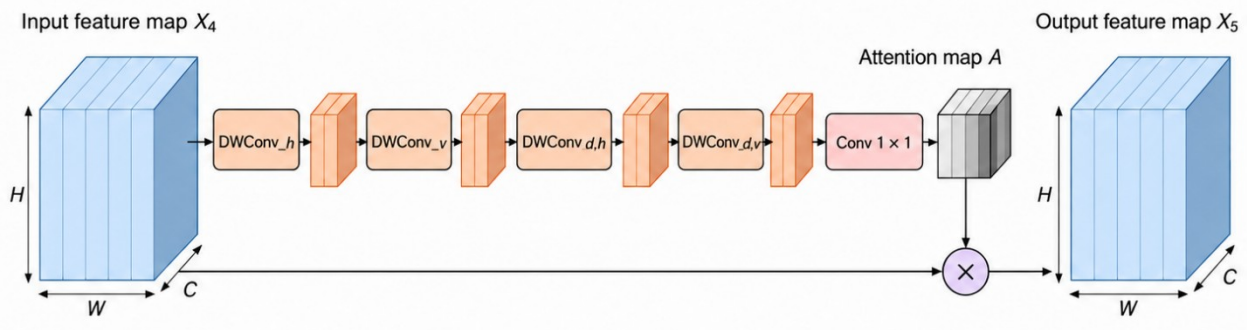


Figure 5. Large separable kernel attention.

3.4 Sign Prediction

3.4.1 Box and Class Estimation

The refined tensor X_5 passes through the detector neck to the class and box prediction branches. The class branch estimates a score for each traffic sign category. The box branch estimates the distances from each feature location to the left, top, right, and bottom edges of a candidate box. These distances are decoded relative to the feature location and mapped to the resized image coordinates. The class scores and box coordinates at the same location form a candidate detection. This two branch design separates category prediction from spatial localization while allowing both branches to use the refined features.

3.4.2 Detection Selection

Candidates below the confidence threshold are discarded. Nonmaximum Suppression (NMS) removes redundant candidates within each class according to box overlap. The inverse resizing and padding transformation maps the selected boxes to the original image. Each result contains a bounding box, class label, and confidence score. The output annotation in Figure 1 illustrates this format rather than a recorded inference result.

The detector training objective supervises category prediction and box localization using the annotated class labels and bounding boxes. The architectural changes concern feature processing and introduce no custom localization objective.

4 Experiment

4.1 Experimental Setup

Experiments were conducted using PyTorch on an NVIDIA RTX 4090 GPU with 24 GB of GPU memory. The system was equipped with 32 GB of RAM and an Intel Core i9 processor. The experiment follows the official GTSRB training and test split defined by Stallkamp et al. [9].

4.1.1 Datasets

We use three datasets: (1) German Traffic Sign Recognition Benchmark (GTSRB), (2) Chinese Traffic Signs (CTS), and (3) Tsinghua-Tencent 100K (TT100K).

The GTSRB dataset contains 51,839 color images in 43 traffic sign classes, including 39,209 training images and 12,630 test images [9]. Images contain an individual sign with some surrounding background and range from 15 × 15 to 250 × 250 pixels. It supplies the training and test data for the parameter study.

The CTS dataset contains 5,998 images across 58 categories with sign annotations [17]. Section 4.2.4 reports dataset-level mAP and recall for CTS, GTSRB, and TT100K.

4.1.2 Algorithms

We compare three recently published traffic sign detection methods against HASAN: (1) Improved YOLOv7 [11]. (2) CPB-YOLOv8 [12]. (3) Tversky-YOLO [13].

4.1.3 Factors

We compared five factors: (1) Kernel configuration, it controls the spatial extent of filtering. (2) Learning rate, it sets the scale of parameter updates. (3) Learning rate scheduler, it adjusts this scale during training. (4) Batch size, it controls the number of training images used per update. (5) Optimizer, it determines the update rule. Each comparison varies one factor while fixing the others. Kernel configuration, learning rate, and scheduler are reported in the main paper. Batch size and optimizer results are provided in Appendix A at the end of this technical report.

4.1.4 Metrics

We use three metrics: (1) mean Average Precision at an Intersection over Union threshold of 0.5 (mAP@0.5), (2) recall, and (3) inference time.

Inference time is the mean prediction duration over ten timed passes after three warmup passes, with inference batch size fixed at 32. We divide the mean test set duration by the number of images and convert seconds to milliseconds. This gives amortized time per image rather than single image latency.

mAP@0.5 is the mean Average Precision calculated across all classes using an Intersection over Union (IoU) threshold of 0.5. A predicted bounding box is considered correct when its IoU with the ground truth box is at least 0.5 and the predicted class is correct. Higher mAP@0.5 indicates better overall detection performance.

Recall is the proportion of actual objects that are correctly detected by the model. It is calculated as the number of true positives divided by the sum of true positives and false negatives. Higher recall indicates that the model misses fewer traffic signs.

4.2 Experimental Results

Each parameter figure presents mAP, recall, and inference time in three side-by-side panels. Solid curves show HASAN, and horizontal dashed lines show reported values for Improved YOLOv7 [11], CPB-YOLOv8 [12], and Tversky-YOLO [13]. Tversky-YOLO recall is unavailable and is omitted from the recall panels. Bold labels identify the reference settings.

4.2.1 Effect of Kernel Configuration

We tested three kernel configurations [mixed 5 × 5 and 3 × 3, all 3 × 3, all 5 × 5]. Only the kernel configuration changed. All 3 × 3 kernels gave the highest mAP, 97.997%, compared with 97.110% for mixed kernels and 97.601% for all 5 × 5 kernels. The best mAP exceeds the reported 96.800% of Tversky-YOLO [13] by 1.197 percentage points, a relative improvement of 1.237%. Recall was 96.482%, 97.522%, and 97.014% for mixed kernels, all 3 × 3 kernels, and all 5 × 5 kernels, respectively. Figure 6 compares mAP, recall, and inference time. Inference times were 0.1397, 0.1186, and 0.1294 ms per image in the same order.

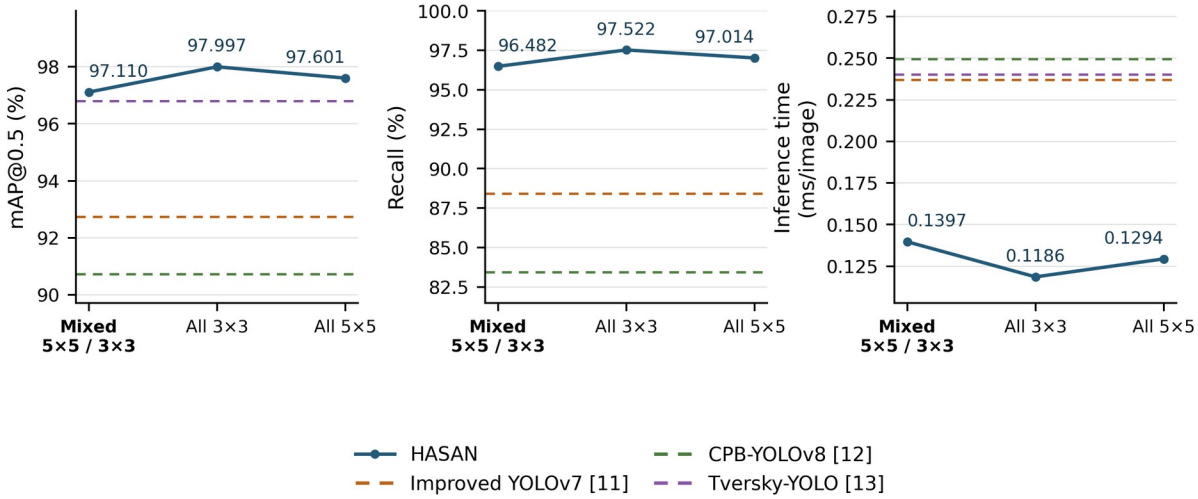


Figure 6. Effect of kernel configuration on mAP, recall, and inference time.

4.2.2 Effect of Learning Rate

We tested five learning rates [0.01, 0.001, 0.0001, 0.00003, 0.00001]. Only the initial learning rate changed. The reference rate of 0.001 gave the highest mAP, 97.110%. The other settings achieved 5.463%, 96.857%, 95.321%, and 86.302%, respectively. Recall was 3.801%, 96.482%, 96.144%, 94.412%, and 84.038% in the same order of learning rates. The poor result at 0.01 indicates that increasing the update scale did not improve this configuration. Inference times were 0.1179, 0.1190, 0.1204, 0.1213, and 0.1188 ms per image in the same order. Figure 7 compares the three metrics.

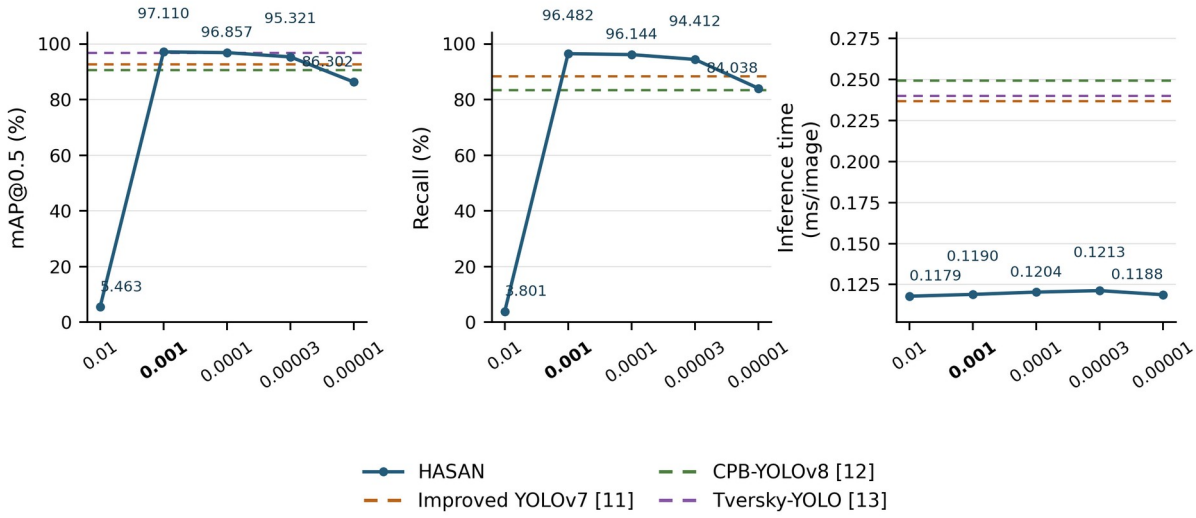


Figure 7. Effect of learning rate on mAP, recall, and inference time.

4.2.3 Effect of Learning Rate Scheduler

We tested three learning rate schedulers [none, ReduceLROnPlateau, CosineAnnealing]. Only the scheduler changed. CosineAnnealing gave the highest mAP, 97.981%, followed by ReduceLROnPlateau at 97.949% and the reference without a scheduler at 97.110%. Recall was 96.482%, 97.390%, and 97.469% for no scheduler, ReduceLROnPlateau, and CosineAnnealing, respectively. Figure 8 compares mAP, recall, and inference time. Inference times were 0.1184, 0.1228, and 0.1285 ms per image in the same order.

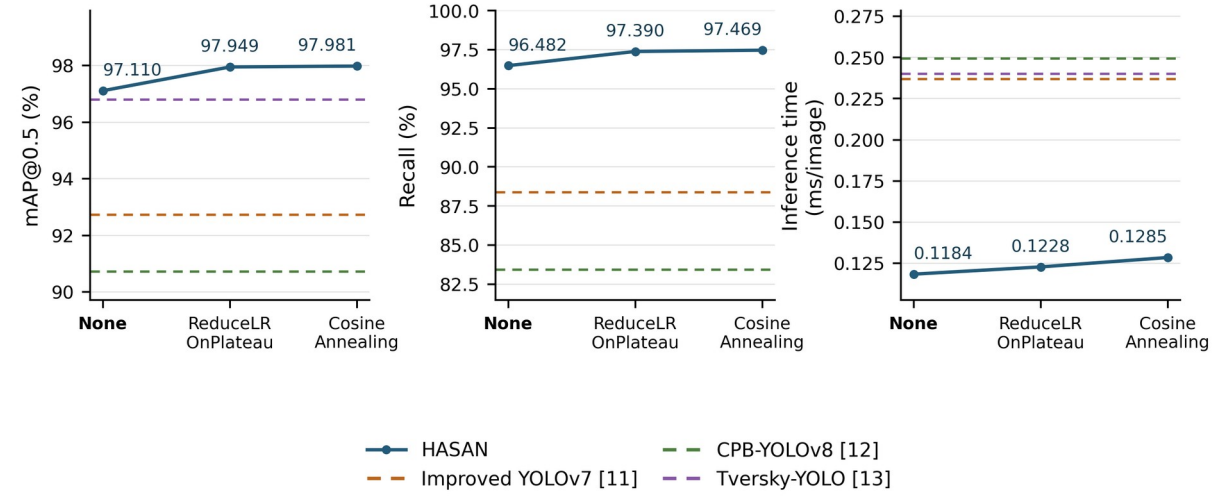


Figure 8. Effect of scheduler on mAP, recall, and inference time.

4.2.4 Effect of Evaluation Dataset

We evaluated HASAN on CTS, GTSRB, and TT100K. Figure 9 presents mAP@0.5 and recall in two side-by-side panels, with the same dataset order on both horizontal axes.

HASAN achieved mAP@0.5 values of 98.225%, 97.997%, and 96.843% on CTS, GTSRB, and TT100K, respectively. The corresponding recall values were 96.782%, 96.314%, and 95.427%. CTS gave the highest mAP and recall among the three evaluated datasets.

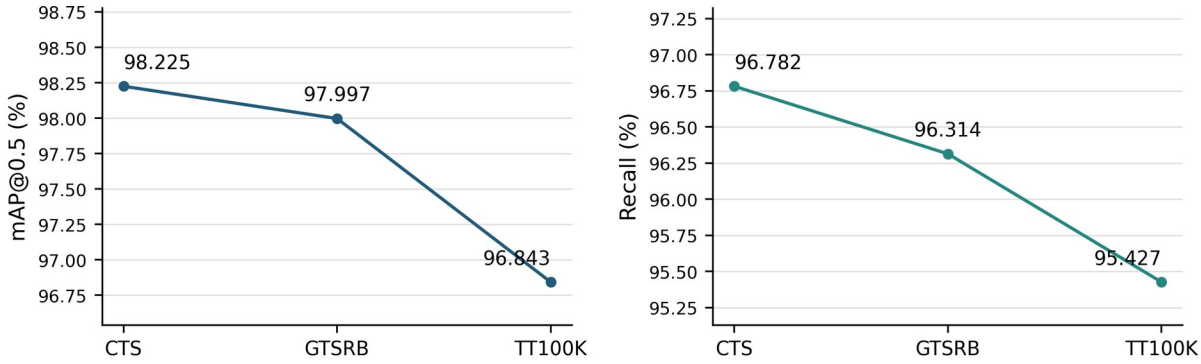


Figure 9. HASAN mAP@0.5 and recall on CTS, GTSRB, and TT100K.

4.3 Experimental Summary

The all 3 × 3 kernel configuration achieved 97.997% mAP@0.5 on GTSRB, compared with the reported 96.800% of Tversky-YOLO [13], the strongest comparison method. The difference is 1.197 percentage points, corresponding to a relative improvement of 1.237%. Learning rate and scheduler comparisons identify separate sources of performance variation. Dataset evaluation yielded mAP@0.5 values of 98.225%, 97.997%, and 96.843% on CTS, GTSRB, and TT100K, with recall of 96.782%, 96.314%, and 95.427%, respectively. Across all five parameter studies, HASAN inference time ranged from 0.1179 to 0.1397 ms per image under the fixed batch protocol.

5 Conclusion

This paper proposes HASAN, a traffic sign detector design combining selective convolution, channel gating, spatial pooling, and spatial attention. Parameter tuning achieved 97.997% mAP@0.5 on GTSRB, compared with 96.800% for the strongest state-of-the-art comparison method. This represents a gain of 1.197 percentage points, or 1.237% relative. Evaluation on CTS, GTSRB, and TT100K further examined mAP and recall across different traffic sign datasets. Future work will investigate novel attention mechanisms and data augmentation strategies to improve detection performance and generalizability.

References

[1] Flores-Calero, M., Astudillo, C. A., Guevara, D., Maza, J., Lita, B. S., Defaz, B., Ante, J. S., Zabala-Blanco, D., & Armingol Moreno, J. M. (2024). Traffic sign detection and recognition using YOLO object detection algorithm: A systematic review. Mathematics, 12(2), 297.

[2] Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. Advances in Neural Information Processing Systems, 28.

[3] Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). Feature Pyramid Networks for object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2117–2125.

[4] Wang, C.-Y., Bochkovskiy, A., & Liao, H.-Y. M. (2023). YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 7464–7475.

[5] Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J., & Ding, G. (2024). YOLOv10: Real-time end-to-end object detection. Advances in Neural Information Processing Systems, 37.

[6] Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., & Chen, J. (2024). DETRs beat YOLOs on real-time object detection. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 16965–16974.

[7] Ahmed, S., Kamal, U., & Hasan, M. K. (2022). DFR-TSD: A deep learning based framework for robust traffic sign detection under challenging weather conditions. IEEE Transactions on Intelligent Transportation Systems, 23(6), 5150–5162.

[8] Luo, Y., Ci, Y., Zhang, H., & Wu, L. (2024). A YOLOv8-CE-based real-time traffic sign detection and identification method for autonomous vehicles. Digital Transportation and Safety, 3(3), 82–91.

[9] Stallkamp, J., Schlipsing, M., Salmen, J., & Igel, C. (2011). The German Traffic Sign Recognition Benchmark: A multi-class classification competition. International Joint Conference on Neural Networks (IJCNN), 1453–1460.

[10] Ertler, C., Mislej, J., Ollmann, T., Porzi, L., Neuhold, G., & Kuang, Y. (2020). The Mapillary Traffic Sign Dataset for Detection and Classification on a Global Scale. European Conference on Computer Vision (ECCV), 68–84.

[11] Small traffic sign recognition method based on improved YOLOv7. (2025). Scientific Reports. https://doi.org/10.1038/s41598-025-88679-w

[12] CPB-YOLOv8: An Enhanced Multi-Scale Traffic Sign Detector for Complex Road Environment. (2025). Information, 16(9), 798. https://doi.org/10.3390/info16090798

[13] Ben-Abbou, T., et al. (2026). Asymmetric similarity aware object detection using Tversky YOLO for fine grained traffic sign recognition. Discover Artificial Intelligence, 6, 1081. https://doi.org/10.1007/s44163-026-02156-5

[14] Chen, J., Kao, S.-H., He, H., Zhuo, W., Wen, S., Lee, C.-H., & Chan, S.-H. G. (2023). Run, don't walk: Chasing higher FLOPS for faster neural networks. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 12021–12031.

[15] Shi, D. (2024). TransNeXt: Robust Foveal Visual Perception for Vision Transformers. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). https://github.com/DaiShiResearch/TransNeXt

[16] Lau, K. W., Po, L.-M., & Rehman, Y. A. U. (2024). Large Separable Kernel Attention: Rethinking the large kernel attention design in CNN. Expert Systems with Applications, 236, 121357.

[17] Yemelyanov, D. (n.d.). Chinese Traffic Signs [Data set]. Kaggle.

[18] HASAN code and supplementary results. Appendix A: Additional parameter results. https://github.com/kotder/HASAN

[19] Das, A., Yang, Y., & Subburaj, V. H. (2025). YOLOv7 for Weed Detection in Cotton Fields Using UAV Imagery. AgriEngineering, 7(10), 313. https://doi.org/10.3390/agriengineering7100313

[20] Intasam, A., et al. (2026). Mammogram Analysis with YOLO Models on an Affordable Embedded System. Cancers, 18(1), 70. https://doi.org/10.3390/cancers18010070

[21] Guemas, E., et al. (2024). Automatic patient-level recognition of four Plasmodium species on thin blood smear by a real-time detection transformer (RT-DETR) object detection algorithm: a proof-of-concept and evaluation. Microbiology Spectrum, 12(2), e01440-23. https://doi.org/10.1128/spectrum.01440-23

Appendix A Additional Parameter Results

Each comparison varies one HASAN training factor. Inference batch size remains fixed at 32. The three side-by-side panels report mAP@0.5, recall, and inference time. Horizontal dashed lines show the comparison methods using the reference numbering in the main paper: Improved YOLOv7 [11], CPB-YOLOv8 [12], and Tversky-YOLO [13]. Tversky-YOLO recall is unavailable and is omitted from the recall panels.

A1 Effect of Batch Size

Settings: 16, 32, 64, 128, 256. mAP@0.5 in the same order: 96.857%, 97.110%, 97.356%, 96.865%, 97.609%. Recall: 96.187%, 96.482%, 96.882%, 96.071%, 97.053%. Inference time: 0.1210, 0.1192, 0.1312, 0.1213, 0.1193 ms per image. Bold chart labels identify the reference setting.

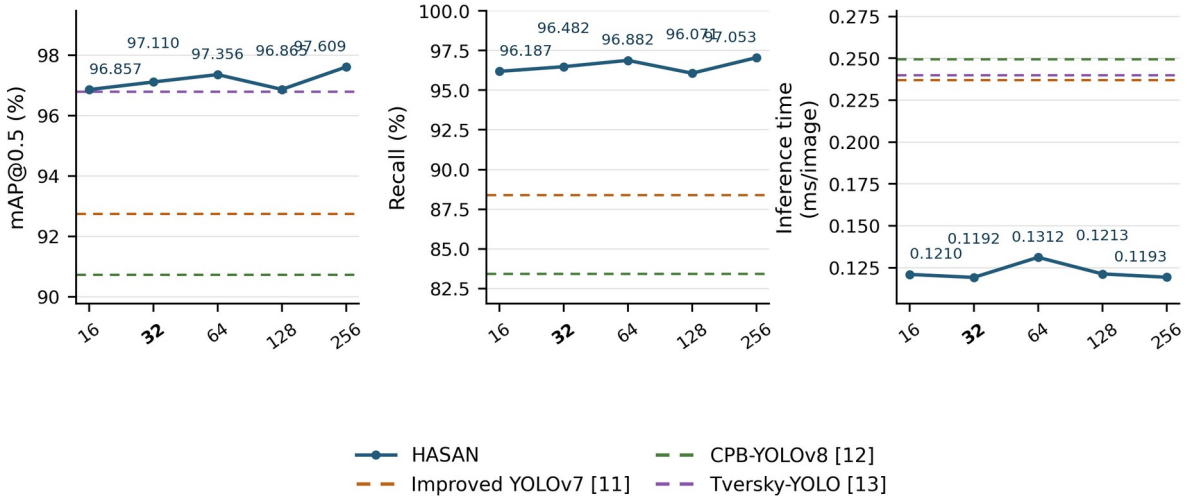


Figure A1. Effect of batch size on mAP@0.5, recall, and inference time.

A2 Effect of Optimizer

Settings: Adam, AdamW, SGD. mAP@0.5 in the same order: 97.110%, 97.664%, 96.603%. Recall: 96.482%, 97.122%, 95.816%. Inference time: 0.1195, 0.1252, 0.1195 ms per image. Bold chart labels identify the reference setting.

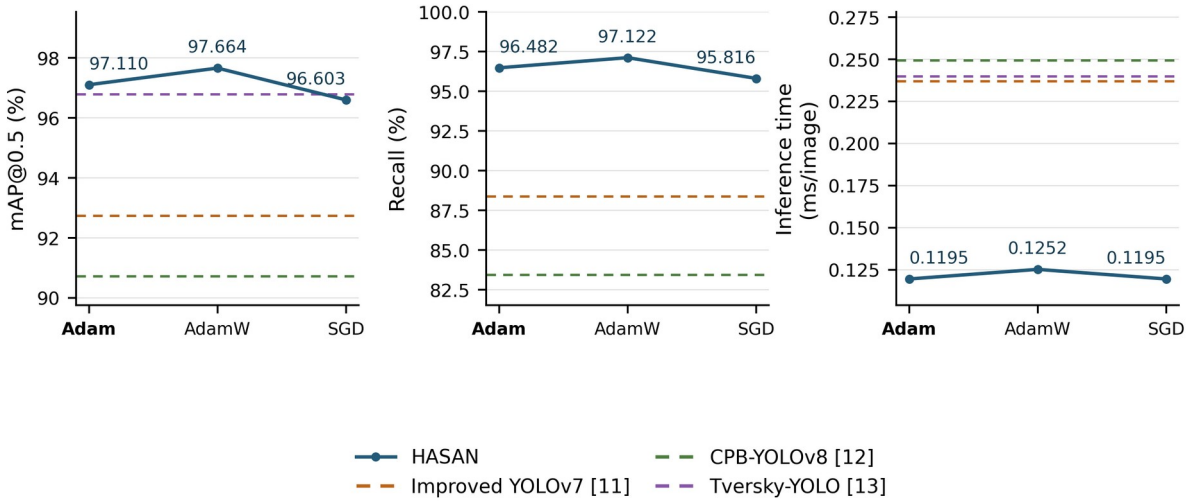


Figure A2. Effect of optimizer on mAP@0.5, recall, and inference time.